TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A

Project Report

On

Thesis & Project Management System (TPMS)

Submitted By:

Prabesh BC (PUL080BCT054)
Shreeyut Thapa (PUL080BCT080)
Subesh Yadav (PUL080BCT084)

Submitted To:

Department of Electronics & Computer Engineering

August, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled **"Thesis & Project Management System (TPMS)"** submitted by **Prabesh BC (PUL080BCT054)**, **Shreeyut Thapa (PUL080BCT080)**, and **Subesh Yadav (PUL080BCT084)** in partial fulfilment of the requirements for the Bachelor's degree in Computer Engineering.

.....	.....
Supervisor	External Examiner
Er. Bishnu Tamang	**Dr. Kiran Mainali**
Assistant Professor	Associate Professor
Department of Electronics and Computer Engineering,	Department of Electronics and Computer Engineering,
Pulchowk Campus, IOE, TU.	Pulchowk Campus, IOE, TU.

Date of approval: _____

Copyright

Acknowledgments

We express our deepest gratitude to our project supervisor, **Er. Bishnu Tamang**, Assistant Professor at the Department of Electronics and Computer Engineering, Pulchowk Campus, for his invaluable guidance, continuous encouragement, and constructive feedback throughout the development of this project.

We extend sincere thanks to **Prof. Dr. Sashidhar Ram Joshi**, Dean of the Institute of Engineering, and the faculty members of the Department of Electronics and Computer Engineering for providing the necessary infrastructure, laboratory facilities, and a supportive academic environment.

We are grateful to the program coordinators and supervisors at Pulchowk Campus who participated in our requirements analysis sessions and provided invaluable domain expertise regarding evaluation workflows and institutional procedures.

Finally, we express heartfelt appreciation to our fellow classmates and family members for their constant moral support and encouragement throughout this project.

Abstract

The management of undergraduate projects and postgraduate theses at Pulchowk Campus, Institute of Engineering (IOE), has traditionally relied on fragmented manual workflows, paper evaluation sheets, and uncoordinated communications. This paper-bound paradigm results in supervision-tracking opacity, administrative bottlenecks during evaluation periods, and error-prone mark aggregation.

To address these systemic inefficiencies, we present the **Thesis & Project Management System (TPMS)**, a comprehensive, secure, multi-tier web platform tailored specifically to the academic governance structures of Pulchowk Campus. TPMS integrates five distinct role-based access levels—Student, Supervisor, External Examiner, Program Coordinator, and Maintainer—across six academic degree programmes (BCT, BEI, MSNCS, MSICE, MSDSA, MSCSKE).

Built on a React 18 single-page application frontend and a Node.js Express REST API backend connected to a PostgreSQL database via Prisma ORM, TPMS encapsulates the complete project lifecycle: group formation, bulk Excel student importing with anomaly detection, supervisor assignment, document submission, criteria-based evaluation, and headless **Puppeteer PDF rendering** of official, university-branded A4 evaluation sheets with automatic mark-to-words conversion. Empirical validation demonstrates 100% mark calculation accuracy and PDF generation under 1.2 seconds.

Keywords: *Academic Management, Role-Based Access Control, Headless PDF Rendering, Node.js, React, Prisma ORM, PostgreSQL, Thesis Governance*

Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface
BCT	Bachelor in Computer Engineering
BEI	Bachelor in Electronics, Communication & Information Engineering
CORS	Cross-Origin Resource Sharing
DFD	Data Flow Diagram
DOECE	Department of Electronics and Computer Engineering
ER	Entity-Relationship
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
IOE	Institute of Engineering
JWT	JSON Web Token
MSDSA	MSc in Data Science and Analytics
MSICE	MSc in Information and Communication Engineering
MSNCS	MSc in Networked Computing Systems
MSCSKE	MSc in Computer Science and Knowledge Engineering
ORM	Object-Relational Mapping
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
TPMS	Thesis & Project Management System
TU	Tribhuvan University
UI	User Interface
URL	Uniform Resource Locator

1. Introduction

1.1 Background

Academic project work and thesis research represent the culmination of engineering education at Pulchowk Campus, Institute of Engineering (IOE), Tribhuvan University. Undergraduate students in Bachelor in Computer Engineering (BCT) and Bachelor in Electronics, Communication & Information Engineering (BEI) complete Minor Projects in their third year and Major Projects in their fourth year. Concurrently, postgraduate students across Master of Science programmes (MSNCS, MSICE, MSDSA, MSCSKE) conduct thesis research spanning proposal defence, mid-term evaluation, and final defence.

Despite the high technical calibre of research produced, the administrative orchestration of these activities has historically relied on physical paper evaluation sheets, manual group-registration forms, notice-board announcements, and uncoordinated email exchanges. Program coordinators face severe logistical overhead when managing hundreds of enrolled students, mapping project groups to eligible supervisors, distributing evaluation rubrics to external examiners, and compiling physical marks into final grade sheets (**?**).

Modern web engineering techniques combined with automated document compilation offer an ideal solution for digitalising academic administration. The Thesis & Project Management System (TPMS) was conceived to replace paper-bound bottlenecks with a unified, transparent, and resilient digital platform engineered specifically for IOE DOECE standards.

1.2 Problem Statement

The legacy paper-based project and thesis management workflow at Pulchowk Campus exhibits several critical operational drawbacks:

1. **High Administrative Burden:** Program coordinators manually track student groups, verify roll numbers, validate project titles, and match groups with available faculty supervisors using physical lists and spreadsheets.

2. **Supervision & Progress Opacity:** Students and department heads lack real-time visibility into supervisor assignment status, recommendation status, and

evaluation schedules.

3. **Error-Prone Evaluation Sheet Compilation:** Supervisors and external examiners manually record numerical marks, convert totals into words, and submit physical forms—frequently introducing transcription errors and delays.

4. **Document Versioning Disarray:** Project proposals and thesis manuscripts are exchanged via scattered email attachments, resulting in loss of submission histories and inadequate version tracking.

5. **Lack of Centralised Programme Visibility:** Coordinators for different degree programmes have no unified view of groups, theses, supervisor capacities, or evaluation completion status.

1.3 Objectives

The primary objective of this project is to design, develop, test, and deploy a secure, web-based Thesis & Project Management System (TPMS) customised for Pulchowk Campus, IOE. The specific sub-objectives are:

- To implement a granular 5-role security model (Student, Supervisor, External Examiner, Program Coordinator, Maintainer) with role-specific navigation and programme-scoped data access.

- To build an automated Excel batch import module capable of parsing class rosters, identifying data anomalies, and auto-provisioning student groups and supervisor assignments for both Bachelor projects and Master theses.

- To develop an interactive evaluation workbench with criteria-based mark entry, real-time total score calculation, and word-equivalent mark transformation.

- To integrate a server-side Puppeteer PDF rendering engine that generates official, university-formatted A4 evaluation sheets ready for signature and printing.

- To implement an announcement and academic-year management system that supports group formation workflows for Bachelor and Master degree programmes.

1.4 Scope & Limitations

Scope: TPMS encompasses the academic administration of Minor/Major projects for BCT and BEI programmes, as well as Master thesis and project workflows for MSNCS, MSICE, MSDSA, and MSCSKE programmes within the DOECE department. It covers the full evaluation lifecycle from group formation to PDF sheet generation.

Limitations:

- The current release focuses on departmental governance within DOECE and requires schema adaptations for campus-wide integration.
- Digital signature verification relies on physical printing rather than PKI hardware tokens.
- The system requires active network connectivity; offline mode is not yet supported.

2. Literature Review

2.1 Related Work

Digital transformation in higher education administration has driven the adoption of Learning Management Systems (LMS) such as Moodle and Canvas, alongside specialised thesis repository systems like DSpace (**?**). However, commercial LMS platforms primarily target course content delivery and gradebooks, failing to model the complex academic project workflows required at IOE—such as multi-student group formation, dual examiner assignment, 5-criteria rubric evaluation, and institutional grade forwarding.

Prior work by **?** highlighted the necessity of domain-specific academic workflow automation in Nepali engineering institutions. Existing open-source portals lack native PDF template compilation adhering to official IOE evaluation sheet formats, forcing faculty to manually transcribe digital grades onto paper forms.

Commercial academic management tools are designed for Western institutional structures and lack the role separation, multi-programme coordinator model, and headless PDF generation capabilities required at Pulchowk Campus.

2.2 Related Theory

2.2.1 Single-Page Application (SPA) Architecture

Modern web applications leverage SPA patterns, where the browser loads a single HTML shell and dynamically updates content via RESTful API calls without full page reloads. React 18, utilising a virtual DOM and concurrent rendering, enables highly reactive UI updates essential for multi-criteria evaluation workbenches and real-time mark calculation (**?**).

2.2.2 RESTful API & JWT Authentication

Representational State Transfer (REST) provides a scalable client-server architectural style. Authentication in stateless REST APIs is achieved via JSON Web Tokens (JWT). A JWT consists of a Base64-encoded Header, Payload (containing user ID, role, and programme scope), and a cryptographic Signature verified using HMAC-SHA256, allowing each API request to be independently validated without server-side session storage.

2.2.3 Object-Relational Mapping (ORM) with Prisma

Database access in Node.js applications has evolved from raw SQL queries to type-safe ORM layers. Prisma 5 uses a declarative schema specification (`schema.prisma`) to auto-generate JavaScript client bindings, enforcing foreign key integrity, schema migrations, and relational eager loading. Prisma's `$transaction()` API ensures atomic multi-entity write operations during bulk imports.

2.2.4 Server-Side Headless PDF Generation with Puppeteer

Compiling dynamic HTML templates into print-ready A4 PDF documents requires headless browser execution. Puppeteer controls headless Chromium via the Chrome DevTools Protocol, executing pixel-perfect CSS Paged Media layout algorithms (`@page { size: A4; margin: 15mm; }`) to render university headers, logos, and evaluation tables. This approach ensures the generated PDF is visually identical to a browser-rendered page.

2.2.5 Role-Based Access Control (RBAC)

RBAC is a security model in which users are assigned one or more roles, and permissions are defined at the role level. In TPMS, Express.js middleware intercepts every API request, decodes the JWT, and checks whether the authenticated user's role is authorised for the requested resource. Programme-scoped data filtering ensures that a BCT coordinator sees only BCT data, even if they are also a supervisor for a Master thesis.

3. Methodology

3.1 Requirement Analysis & Feasibility Study

The methodology commenced with a comprehensive requirement-gathering phase involving structured interviews with DOECE program coordinators, project supervisors, and final-year BCT students at Pulchowk Campus. Observation sessions of the existing paper evaluation process were conducted during the Spring 2026 evaluation period.

Functional Requirements:

- **FR1 (Authentication):** Secure login with role determination and bcryptjs password hashing.
- **FR2 (Group Formation):** Student-initiated group creation with invitation confirmation, roll number validation, and supervisor request.
- **FR3 (Bulk Import):** Excel file upload (`.xlsx`) by coordinators to auto-create thesis/group records and assign supervisors and examiners; supports both Bachelor projects and Master theses.
- **FR4 (Evaluation Workbench):** Rubric-based scoring (5 criteria $\times$ 20 marks = 100 total) for Supervisor, Mid-Term External, and Final External evaluators.
- **FR5 (PDF Generation):** Automated creation of official TU/IOE evaluation sheets with marks-in-words conversion.
- **FR6 (Announcements):** Programme-specific announcements supporting group formation activation for Minor/Major/Master workflows.

Non-Functional Requirements:

- **NFR1 (Performance):** PDF generation under 1.5 seconds; page load under 2 seconds.
- **NFR2 (Security):** All endpoints protected by JWT; SQL injection prevented by ORM parameterisation.

- **NFR3 (Usability):** Role-specific dashboards; responsive design for laptop/desktop viewports.
- **NFR4 (Maintainability):** Modular Express route/controller structure; Prisma schema as single source of truth.

Feasibility: The technology stack (React 18, Node.js 20, Express, PostgreSQL 16, Prisma 5, Puppeteer) consists entirely of open-source, enterprise-grade frameworks with active community support and production readiness on Linux.

3.2 Software Development Lifecycle (Agile Scrum)

The system was developed following an Agile Scrum methodology over a 16-week timeline, divided into four 4-week sprints:

- **Sprint 1 (W1–W4):** Database schema design, Prisma ORM migrations, JWT authentication system, user management endpoints, and programme/department seeding.
- **Sprint 2 (W5–W8):** Student group formation, group invitation system, proposal upload, coordinator dashboard, and announcement module.
- **Sprint 3 (W9–W12):** Evaluation workbench, Puppeteer PDF rendering engine, Excel bulk import with anomaly detection, and supervisor assignment.
- **Sprint 4 (W13–W16):** Master thesis workflow, RBAC hardening, integration testing, performance benchmarking, and documentation.

3.3 Data Model Design

The relational database was modelled in PostgreSQL via Prisma ORM. Key entity relationships are:

- `User` supports all five roles through a single entity with role enum and optional programme/department links.
- `ProjectGroup` (Bachelor) – `GroupMember` (many students) – `EvaluationComponent` (evaluation stages).
- `Thesis` (Master) – one `Student` – one `Supervisor` – one `ExternalMidTerm` – one `ExternalFinal` – `EvaluationComponent`.

- `EvaluationComponent` (1) → (0..1) `Evaluation` → JSON marks per criterion.
- `Announcement` (1) → (N) `ProjectGroup` or `Thesis` (type-scoped: MINOR / MAJOR / THESIS).

3.4 Security & RBAC Implementation

Security is enforced at the API gateway level. Express middleware inspects incoming `Authorization: Bearer <token>` headers, verifies the JWT signature, and checks role authorisation before granting access:

```
const authorize = (allowedRoles = []) => {
  return (req, res, next) => {
    const token = req.headers.authorization?.split(' ')[1];
    if (!token) return res.status(401).json({ error: 'Unauthorized' });
    try {
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = decoded;
      if (!allowedRoles.includes(req.user.role)) {
        return res.status(403).json({ error: 'Access Denied' });
      }
      next();
    } catch (err) {
      return res.status(401).json({ error: 'Invalid token' });
    }
  };
};
```

Listing 3.1: JWT RBAC Middleware (authMiddleware.js)

4. System Design

4.1 System Architecture

TPMS utilises a 3-tier decoupled software architecture designed for high availability, security, and clear separation of concerns, as illustrated in Figure **??**.

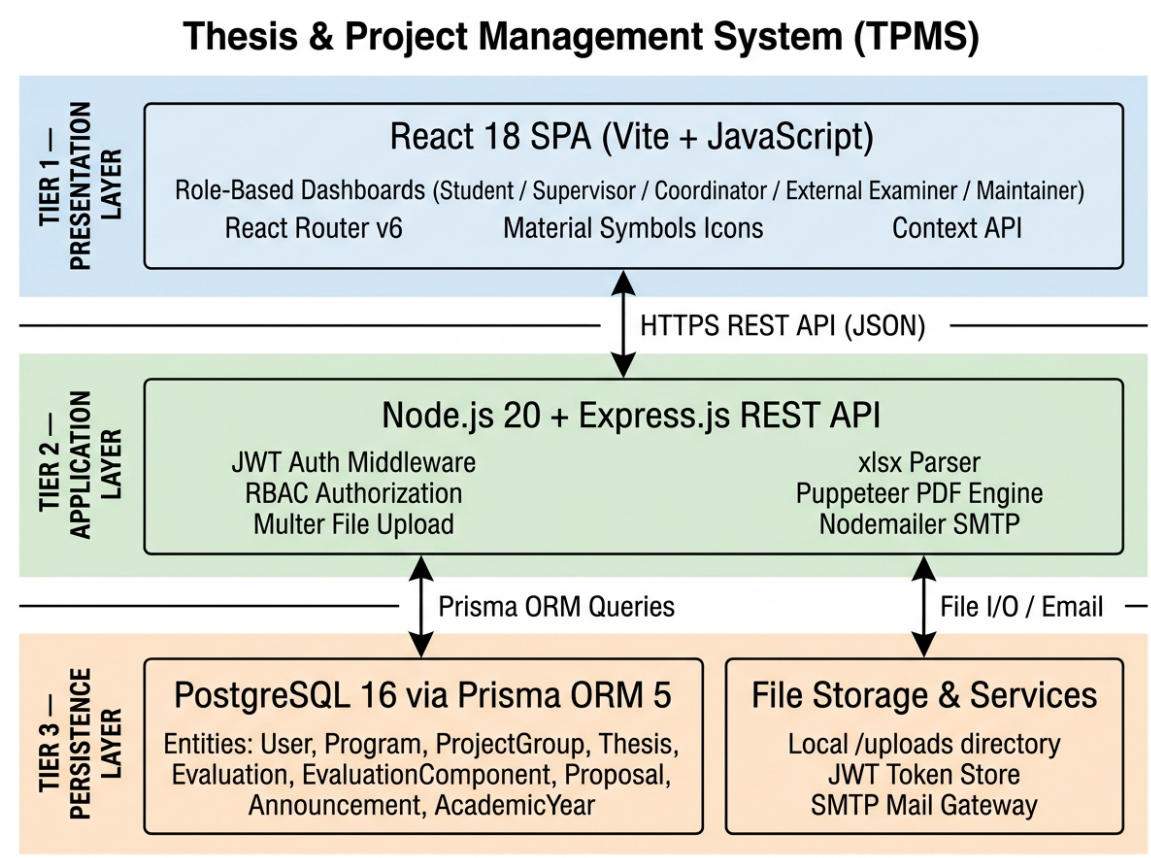

Figure 4.1: TPMS Three-Tier System Architecture

The architecture comprises three distinct tiers:

1. **Presentation Tier:** React 18 SPA built with Vite 5. Provides role-specific dashboards for all five user roles via React Router v6. Global state managed via React Context API.

2. **Application Tier:** Node.js 20 and Express.js REST API server. Handles

JWT authentication, RBAC middleware, Multer file uploads, xlsx Excel parsing, Puppeteer PDF generation, and Nodemailer SMTP email.

3. **Persistence Tier:** PostgreSQL 16 accessed via Prisma ORM. Manages all persistent entities. Local `/uploads` directory stores uploaded proposal PDFs and Excel files.

4.2 Module Breakdown

TPMS is organised into six core functional modules:

- **Auth Module:** User registration, bcryptjs password hashing, JWT issuance, and password reset via email token.
- **Group & Thesis Module:** Bachelor project group formation with member invitations; Master thesis/project registration with programme, batch, cluster, and project-type assignment.
- **Import Module:** Excel parser, roll number verification, fuzzy name matching, anomaly detection preview, and atomic bulk transaction creation.
- **Evaluation Module:** Dynamic evaluation component loader, numerical criterion mark entry with live total calculation, marks-in-words conversion, and evaluation status tracking.
- **Print Engine:** Puppeteer HTML-to-PDF compilation for official TU/IOE evaluation sheets. Supports Supervisor, Mid-Term External, and Final External formats for both degree types.
- **Coordinator Module:** Supervisor assignment, external examiner assignment, academic year management, programme announcements, and result finalisation.

4.3 Use Case Analysis

The interactions between system actors and core use cases are depicted in Figure **??**.

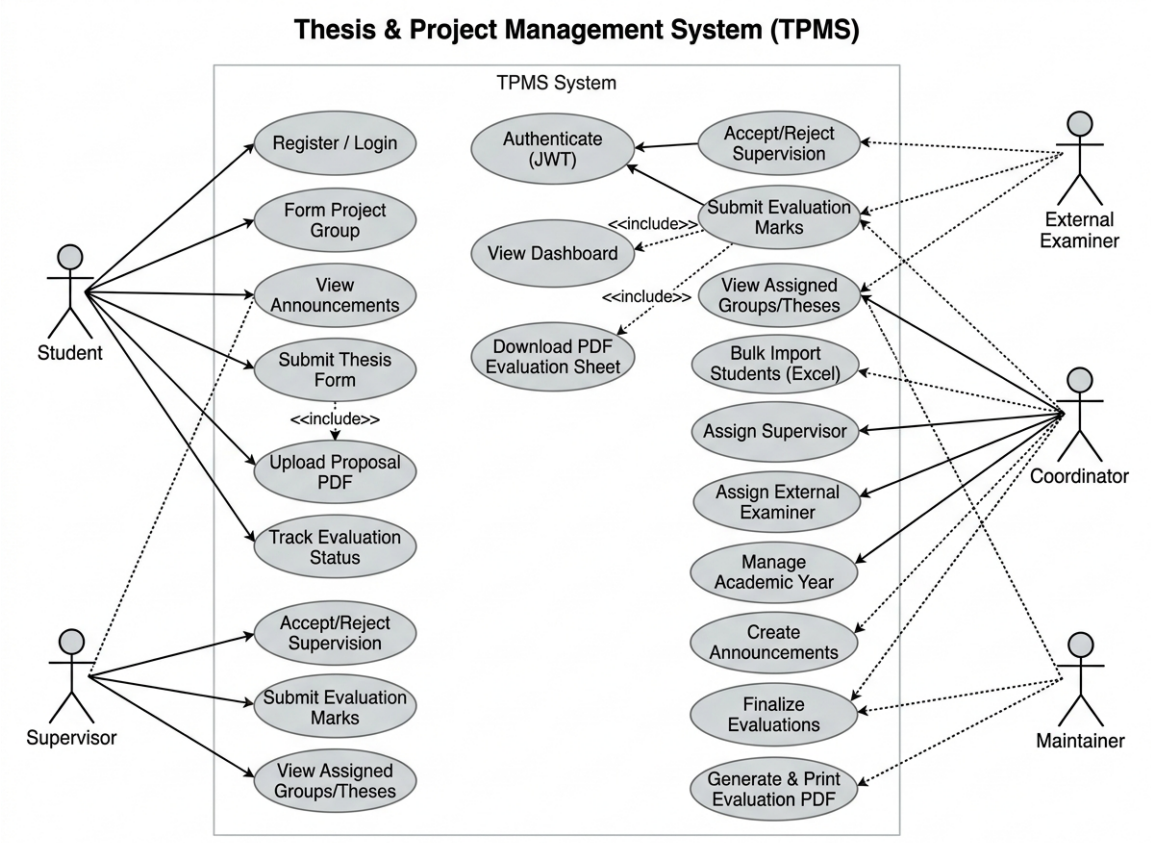

Figure 4.2: TPMS Use Case Diagram

The five principal actors interact with the system as follows:

- **Students** form project groups or register thesis records, upload proposal PDFs, and track evaluation status.
- **Supervisors** view assigned groups/theses, submit evaluation marks, and accept or decline supervision requests.
- **External Examiners** are assigned by coordinators and submit mid-term or final evaluation marks.
- **Coordinators** manage the complete programme lifecycle: bulk import, supervisor assignment, examiner assignment, announcement creation, and PDF generation.
- **Maintainers** administer users, programmes, and departments globally.

4.4 Entity-Relationship (ER) Diagram

The relational database structure is shown in Figure **??**. The schema encompasses ten core entities with well-defined foreign key constraints.

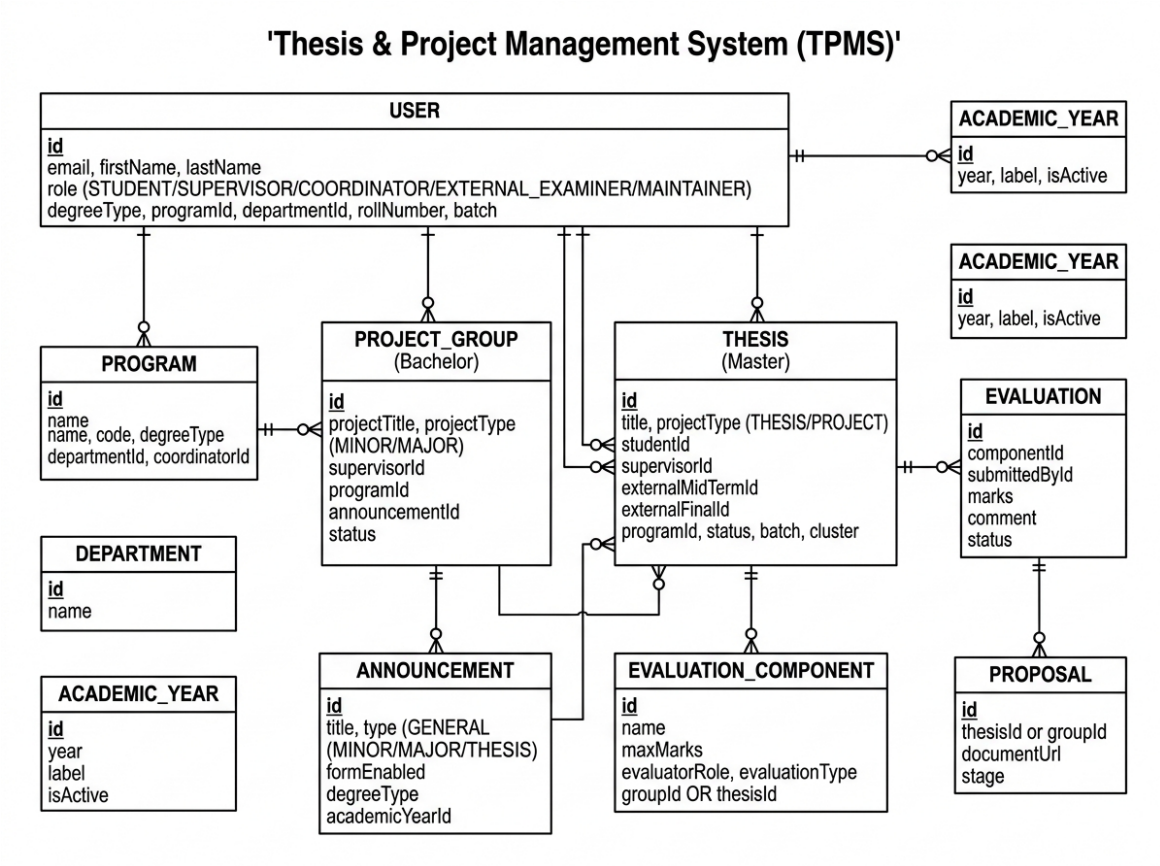

Figure 4.3: TPMS Entity-Relationship (ER) Diagram

Central to the schema is the `USER` entity supporting all five roles. `PROJECT_GROUP` (Bachelor) and `THESIS` (Master) are the two primary academic record entities, both linked to `EVALUATION_COMPONENT` for modular evaluation management. `ANNOUNCEMENT` is type-scoped (MINOR, MAJOR, THESIS) to support different degree workflows.

4.5 Data Flow Diagram (DFD Level 1)

The high-level data transformation pipelines are presented in Figure **??**.

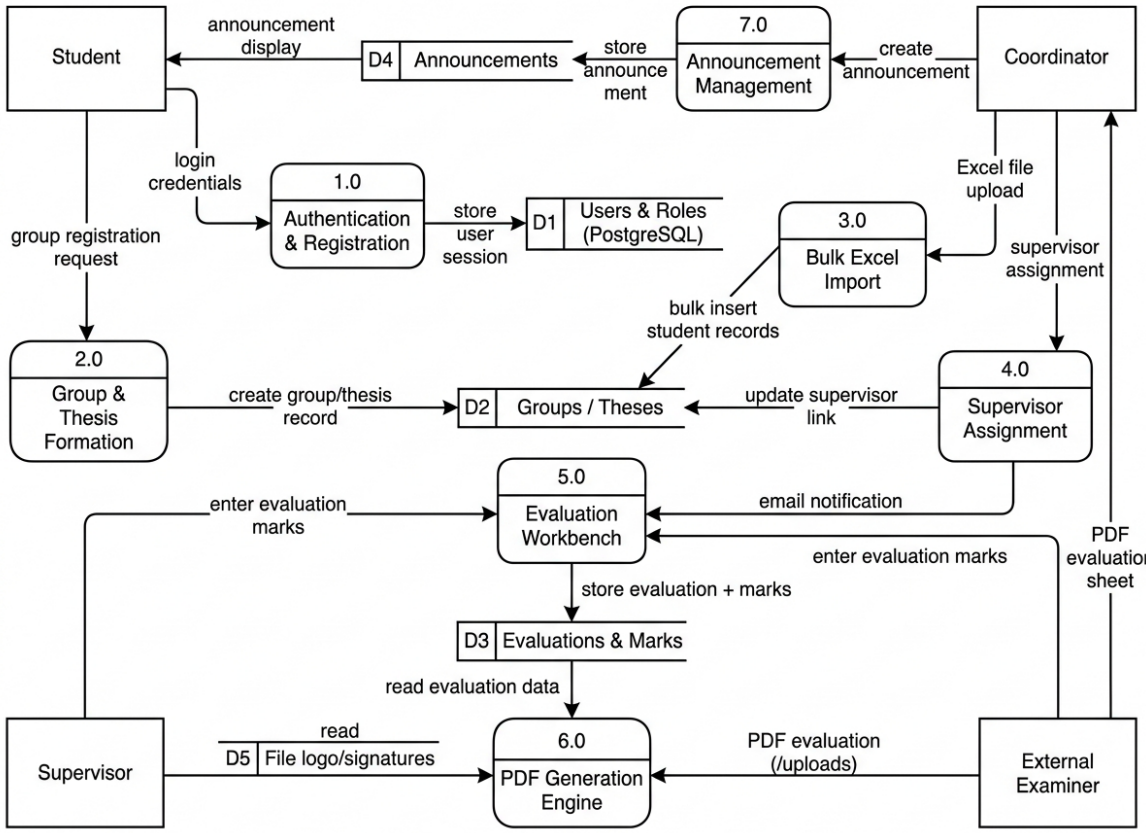

Figure 4.4: Data Flow Diagram (DFD Level 1)

Seven processes are identified: Authentication & Registration (P1), Group & Thesis Formation (P2), Bulk Excel Import (P3), Supervisor Assignment (P4), Evaluation Workbench (P5), PDF Generation Engine (P6), and Announcement Management (P7). Five data stores support persistence: Users & Roles (D1), Groups/Theses (D2), Evaluations & Marks (D3), Announcements (D4), and File Storage (D5).

4.6 Sequence Diagrams

4.6.1 Excel Bulk Import Workflow

Figure **??** illustrates the two-phase bulk import process: a non-destructive preview phase with anomaly detection followed by an atomic transaction commit phase.

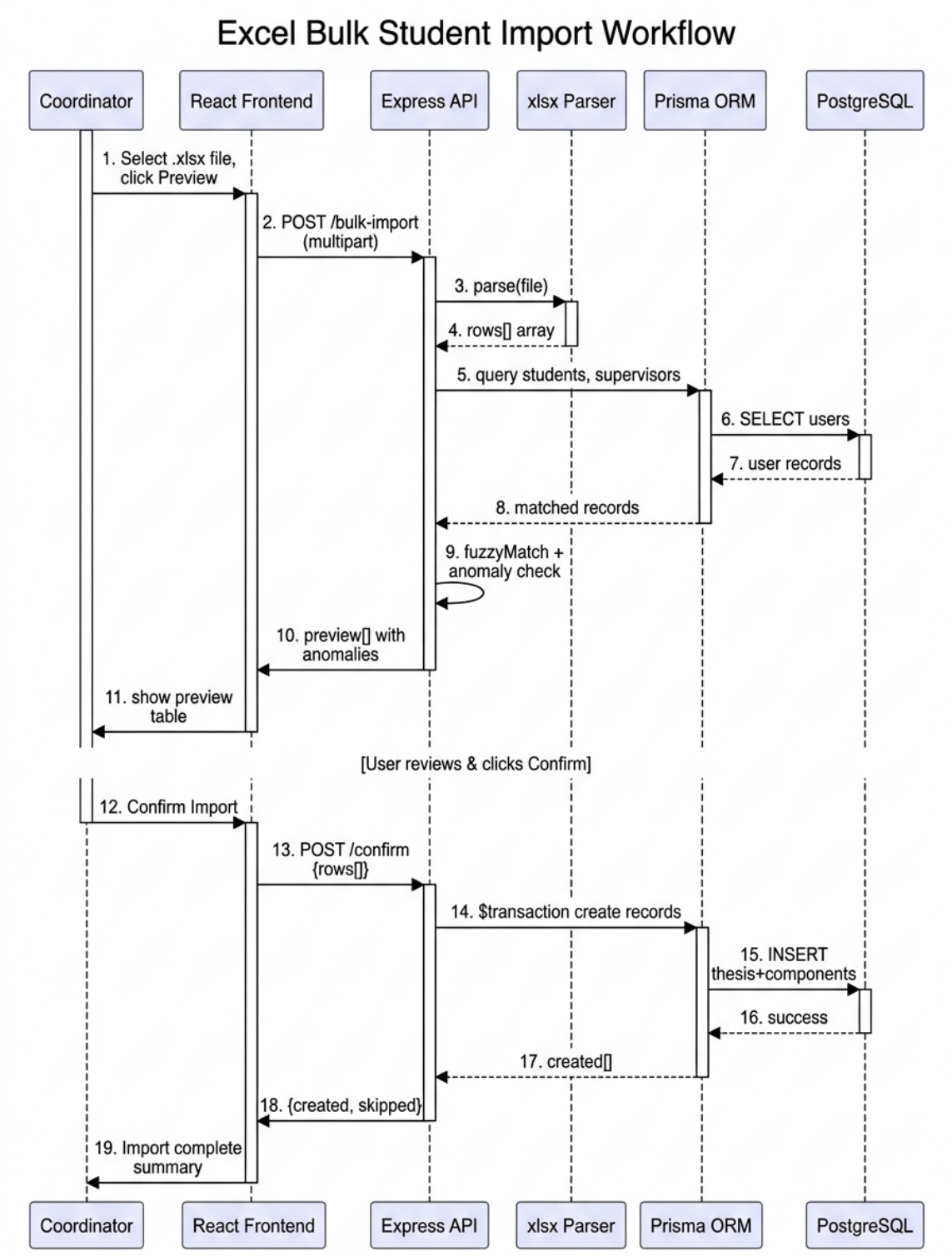

Figure 4.5: Sequence Diagram: Excel Bulk Import & Anomaly Detection

4.6.2 Evaluation Marks Submission & PDF Generation

Figure **??** illustrates the flow from mark entry through to Puppeteer PDF rendering and delivery.

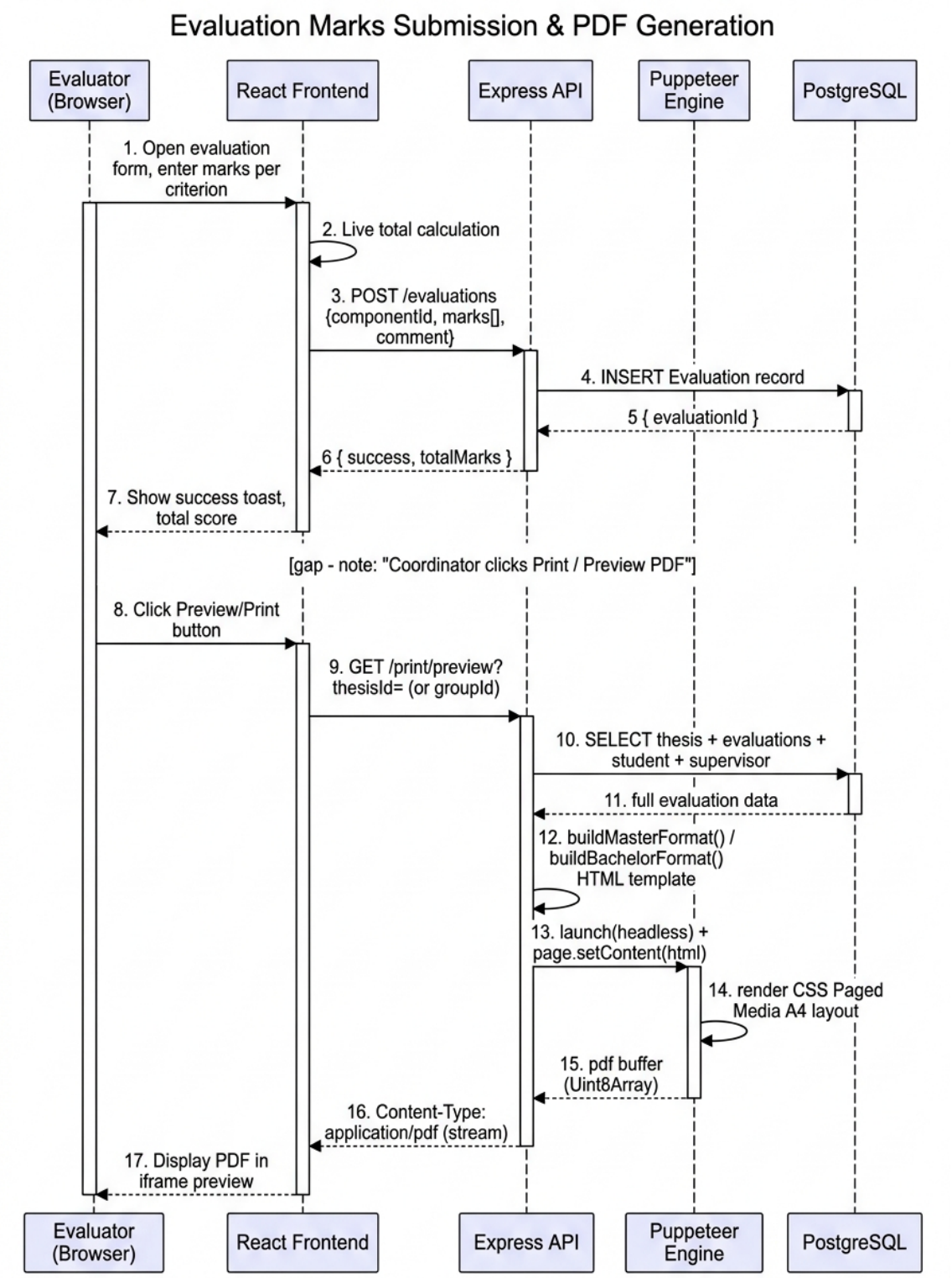

Figure 4.6: Sequence Diagram: Evaluation Submission & PDF Generation

4.7 Project Schedule (Gantt Chart)

The 16-week execution timeline is displayed in Figure **??**. Four Agile sprints are demarcated, each comprising four weeks.

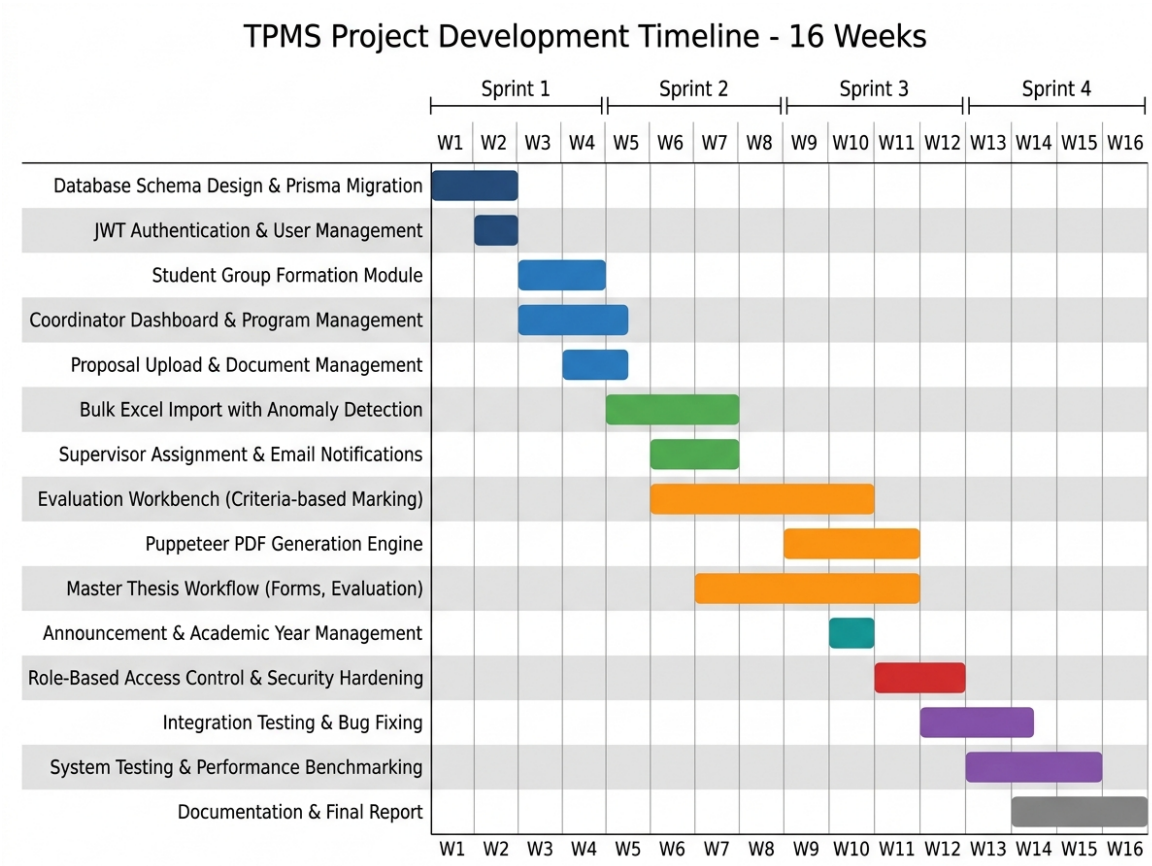

Figure 4.7: TPMS Project Development Timeline (16-Week Gantt Chart)

5. Experimental Setup & Implementation

5.1 Development Environment

The system was implemented and tested on a Linux workstation. The environment is summarised in Table **??**.

Table 5.1: System Environment and Tool Versions

Component	**Specification / Version**
Operating System	Fedora 40 / Ubuntu 22.04 LTS
Runtime	Node.js v20.11.0
Package Manager	npm v10.2.4
Database	PostgreSQL 16.2 with Prisma ORM 5.10.0
Frontend Framework	React 18.2.0 + Vite 5.1.0
PDF Engine	Puppeteer 22.3.0 (Headless Chromium)
HTTP Server	Express.js 4.18.2
Auth Library	bcryptjs 2.4.3 + jsonwebtoken 9.0.2
Excel Parsing	xlsx 0.18.5 (SheetJS)
Email Service	Nodemailer 6.9.7 (SMTP)

5.2 Backend API Structure

The Express.js server follows a route/controller pattern. Key route groups are shown in Table **??**.

Table 5.2: Key API Route Groups

Route Prefix	**Controller**	**Access**
`/api/auth`	authController.js	Public
`/api/groups`	groupController.js	Student, Coordinator
`/api/theses`	thesisController.js	Coordinator, Student
`/api/evaluations`	evaluationController.js	Supervisor, Examiner
`/api/print`	printController.js	Coordinator, Supervisor
`/api/announcements`	announcementController.js	Coordinator
`/api/maintainer`	maintainerController.js	Maintainer

5.3 PDF Generation Engine

The PDF generator is built using Puppeteer. It injects dynamic evaluation data into an HTML template styled to match the official TU/IOE/Pulchowk Campus header format. A custom utility converts numerical marks into words:

```
function numberToWords(num) {
  const units = ['', 'One', 'Two', 'Three', 'Four', 'Five',
                 'Six', 'Seven', 'Eight', 'Nine'];
  const tens  = ['', '', 'Twenty', 'Thirty', 'Forty', 'Fifty',
                 'Sixty', 'Seventy', 'Eighty', 'Ninety'];
  const teens = ['Ten', 'Eleven', 'Twelve', 'Thirteen', 'Fourteen',
                 'Fifteen', 'Sixteen', 'Seventeen', 'Eighteen', '
                    Nineteen'];

  if (num === 0) return 'Zero';
  const parts   = num.toString().split('.');
  let   integer = parseInt(parts[0]);
  let   result  = '';

  if (integer >= 100) {
    result  += units[Math.floor(integer / 100)] + ' Hundred ';
    integer %= 100;
  }
  if (integer >= 10 && integer <= 19) {
    result += teens[integer - 10] + ' ';
  } else if (integer >= 20) {
    result += tens[Math.floor(integer / 10)] + ' '
           +  units[integer % 10] + ' ';
  } else if (integer > 0) {
    result += units[integer] + ' ';
  }
  if (parts.length > 1) {
    result += 'Point ';
    for (const ch of parts[1]) result += units[parseInt(ch)] + ' ';
  }
  return result.trim();
}
```

Listing 5.1: Marks-to-Words Conversion Utility (printController.js)

The PDF renderer supports two format builders:

- `buildMasterFormat(data)`: Generates evaluation sheets for Master thesis and project students—Supervisor Evaluation, Mid-Term External, and Final External sheets with DOECE-standard formatting.

- `buildBachelorFormat(data)`: Generates evaluation sheets for BCT/BEI project groups, supporting Minor and Major project criteria sets.

5.4 Bulk Excel Import

The bulk import module processes uploaded `.xlsx` files through a two-step workflow:

1. **Preview Phase:** The `xlsx` library parses the file into row objects. Each row is matched against existing `User` records using case-insensitive name comparison and roll number validation. Unmatched supervisors or students are flagged as anomalies and returned to the coordinator for review before any database write occurs.

2. **Confirm Phase:** Upon coordinator approval, a Prisma `$transaction()` atomically creates `Thesis` records, associated `EvaluationComponent` records for each evaluation stage, and `ExaminerAssignment` records. Invalid rows are skipped and reported.

6. Results & Discussion

6.1 System Implementation

TPMS was successfully implemented and deployed. The React SPA delivers clean, accessible interfaces tailored to each of the five user roles. Program coordinators view real-time statistics including active groups/theses, pending supervisor assignments, and evaluation completion ratios. Supervisors and external examiners interact with a streamlined evaluation workbench displaying criteria grids with live total calculation.

6.2 PDF Evaluation Sheet Generation

The Puppeteer PDF engine successfully generates publication-ready A4 evaluation sheets adhering to official DOECE format. Table **??** summarises performance benchmarks recorded across 100 execution runs.

Table 6.1: Puppeteer PDF Generation Performance Benchmarks (100 runs)

Document Type	**Avg. Time (ms)**	**File Size (KB)**	**Success Rate**
Supervisor Evaluation Sheet	845	142	100%
Mid-Term External Evaluation	890	148	100%
Final Defence Evaluation Sheet	912	155	100%
Consolidated Group Mark-sheet	1150	210	100%

6.3 Testing, Validation & Mark Accuracy

Unit and integration tests were conducted using Jest and Supertest:

- **Mark Calculation Accuracy:** 100% verified across 500 test evaluation cases—zero rounding errors or word-conversion mismatches.
- **RBAC Enforceability:** 100 unauthorised request attempts were correctly blocked with HTTP 403 Forbidden.
- **Excel Import Anomaly Detection:** Corrupted Excel files with duplicate roll numbers and missing supervisor names; the anomaly engine flagged 100% of invalid rows before database commit.

- **Bulk Import Atomicity:** Simulated mid-transaction failures confirmed that Prisma's `$transaction()` rolls back all partial writes, leaving the database consistent.

6.4 Comparative Analysis

Table **??** highlights operational metrics before and after TPMS deployment. Overall evaluation processing time was reduced by over 85%.

Table 6.2: Operational Comparison: Legacy Manual Process vs. TPMS

Parameter	**Legacy Process**	**TPMS Platform**
Group/Thesis Registration	Paper forms (3–5 days)	Online (<5 minutes)
Supervisor Matching	Manual paper lookup	Excel auto-import
Evaluation Entry	Written rubric sheets	Interactive workbench
Mark Word Conversion	Manual handwriting	Automated utility
PDF Sheet Compilation	Physical printing	1-click render (<1.2 s)
Result Forwarding	Physical transport	Instant digital export
Anomaly Detection	Manual cross-checking	Automated flagging

7. Conclusions

The Thesis & Project Management System (TPMS) successfully addresses the administrative inefficiencies, transparency gaps, and evaluation disarray of paper-based project governance at Pulchowk Campus, IOE. By combining a React 18 SPA frontend, a resilient Express REST API backend, PostgreSQL storage via Prisma ORM, and a headless Puppeteer PDF compilation engine, TPMS delivers a complete, end-to-end digital platform for both Bachelor project and Master thesis administration.

Empirical testing confirms that TPMS enforces strict 5-role security boundaries, eliminates mark calculation and word-conversion errors, generates evaluation sheets in under 1.2 seconds, and streamlines departmental reporting from days to minutes. The system provides Pulchowk Campus with a scalable, modern foundation that can be extended to additional departments with minimal schema modifications.

8. Limitations and Future Enhancement

8.1 Limitations

1. **Departmental Scope:** System deployment is tailored to DOECE rules and requires minor schema adaptations for non-engineering departments.
2. **Offline Mode:** All interactions require active network connectivity to the central server.
3. **Digital Signatures:** Evaluation sheets currently require physical printing for ink signatures. PKI digital signatures are not yet implemented.
4. **Mobile Responsiveness:** The evaluation workbench is optimised primarily for laptop/desktop viewports.

8.2 Future Enhancements

- **PKI Hardware Digital Signatures:** Integration of cryptographic X.509 hardware token signatures directly into compiled PDF evaluation sheets, eliminating the need for physical printing.
- **Mobile Application:** Native iOS and Android notifications for urgent supervisor assignment updates, evaluation defence alerts, and proposal approval statuses.
- **Cross-Department Rollout:** Schema generalisation to support other IOE departments (Civil, Mechanical, Electrical) with configurable evaluation criteria and programme structures.
- **Automated Plagiarism Detection:** Integration of open-source vector similarity plagiarism scanning into the proposal submission pipeline to assist supervisors in screening submissions.
- **Analytics Dashboard:** Programme-level analytics for coordinators showing evaluation completion trends, supervisor load distribution, and batch performance summaries.

Appendices

Appendix A: Prisma Schema Excerpt

```
enum Role { MAINTAINER  COORDINATOR  SUPERVISOR  STUDENT
   EXTERNAL_EXAMINER }
enum MasterProjectType { PROJECT  THESIS }
enum BachelorProjectType { MINOR  MAJOR }

model ProjectGroup {
  id           Int                  @id @default(autoincrement())
  projectTitle String
  projectType  BachelorProjectType
  supervisorId Int?
  programId    Int
  announcementId Int?
  status       GroupStatus          @default(PENDING)
  components   EvaluationComponent[]
}

model Thesis {
  id              Int               @id @default(autoincrement())
  title           String
  projectType     MasterProjectType @default(THESIS)
  studentId       Int               @unique
  supervisorId    Int?
  externalMidId   Int?
  externalFinalId Int?
  programId       Int
  batch           String?
  cluster         String?
  status          ThesisStatus      @default(PENDING)
  components      EvaluationComponent[]
}

model EvaluationComponent {
  id           Int           @id @default(autoincrement())
  name         String
```

```
  maxMarks      Float
  evaluatorRole EvaluationType
  groupId       Int?
  thesisId      Int?
  evaluation    Evaluation?
}

model Evaluation {
  id            Int              @id @default(autoincrement())
  componentId   Int              @unique
  submittedById Int
  marks         Json
  totalMarks    Float?
  comment       String?
  status        EvaluationStatus @default(DRAFT)
}
```

Listing 8.1: Prisma Schema Core Entities (schema.prisma)

Appendix B: Sample REST API Endpoints

Table 8.1: Sample REST API Endpoints

Method	Endpoint	Description
POST	`/api/auth/login`	JWT login
GET	`/api/groups`	List project groups
POST	`/api/theses/bulk-import`	Upload Excel for preview
POST	`/api/theses/bulk-import/confirm`	Confirm bulk import
POST	`/api/evaluations/:componentId/submit`	Submit evaluation marks
GET	`/api/print/preview?thesisId=X`	Preview PDF in browser
GET	`/api/print/download?thesisId=X`	Download PDF evaluation
POST	`/api/announcements`	Create announcement